



Basi di Dati
Progetto A.A. 2025/2026

TITOLO DEL PROGETTO

Matricola

Nome e Cognome

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti.....	3
3. Progettazione concettuale.....	4
4. Progettazione logica.....	5
5. Progettazione fisica.....	7

1. Descrizione del Minimondo

- | | |
|----|---|
| 1 | <p>Si vuole realizzare il sistema informativo di una società di trasporto taxi. Ciascun tassista può liberamente iscriversi al servizio, mettendo a propria disposizione il proprio veicolo. All'atto dell'iscrizione al servizio, l'autista deve registrare il proprio nome e cognome, il numero di patente, la targa del proprio veicolo ed il numero di carta di credito con cui pagare il costo del servizio. Deve inoltre comunicare quanti posti ha a disposizione il suo veicolo.</p> <p>Gli utenti che utilizzano il sistema, all'atto della registrazione, devono fornire il proprio nome e cognome, un numero di telefono ed un numero di carta di credito per poter pagare le corse.</p> |
| 10 | <p>Quando hanno bisogno di un taxi, gli utenti indicano l'indirizzo di partenza e l'indirizzo di destinazione. Un tassista ha la possibilità di visualizzare tutte le richieste di corse ancora attive e di selezionare una corsa per assegnarsela. Se dopo 2 minuti dalla richiesta nessun tassista ha accettato la corsa, la richiesta viene marcata come scaduta.</p> <p>Al termine della corsa il tassista indica l'importo mostrato dal tassametro. Il sistema <u>calcolerà</u> automaticamente la durata complessiva della corsa.</p> |
| 21 | <p>I gestori del servizio ricevono un compenso pari al 3% dell'importo di tutte le corse pagate ai tassisti. Per questo motivo, possono generare un report che riporti, in forma aggregata per ciascun tassista: quante corse sono state accettate, l'importo complessivo del guadagno del tassista, l'importo complessivo della commissione associato al tassista. Per le corse per cui non è ancora stata riscossa la percentuale, il sistema calcola la percentuale del 3%. I gestori possono indicare che l'importo è stato riscosso dalla carta di credito del tassista.</p> |

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
4	autista	tassista	Compare solo alla riga 4 in sostituzione di "tassista", usato in tutto il resto della specifica. Si standardizza a "tassista" per uniformità.
11,16	utenti	cliente	I termine "utenti" è generico; si adotta "cliente" per indicare gli utenti finali che richiedono corse, distinguendoli dai tassisti e dai gestori.
8	posti	capacità del veicolo	Il termine "posti" è informale. Si adotta "capacità" come attributo del veicolo, più preciso e coerente con il dizionario dei dati.
19, 20	richiesta di corsa	richiesta	Nella specifica "richiesta di corsa" e "corsa" vengono usati come sinonimi, mentre rappresentano concetti distinti. Si abbrevia in "richiesta" per distinguerla nettamente da "corsa" ed evitare confusione nel resto del documento.
28, 33, 35–36	compenso/percentuale del 3%	commissione	La specifica usa tre termini per lo stesso concetto: "compenso", "commissione", "percentuale del 3%". Si standardizza a "commissione", termine già presente nella specifica e più preciso.

Specifica disambiguata

Si vuole realizzare il sistema informativo di una società di trasporto taxi. Ciascun tassista può liberamente iscriversi al servizio, mettendo a propria disposizione il proprio veicolo. All'atto dell'iscrizione al servizio, il tassista deve registrare il proprio nome e cognome, il numero di patente, la targa del proprio veicolo ed il numero di carta di credito con cui pagare il costo del servizio. Deve inoltre comunicare la capacità del veicolo.

I clienti che utilizzano il sistema, all'atto della registrazione, devono fornire il proprio nome e cognome, un numero di telefono ed un numero di carta di credito per poter pagare le corse.

Quando hanno bisogno di un taxi, i clienti indicano l'indirizzo di partenza e l'indirizzo di destinazione. Un tassista ha la possibilità di visualizzare tutte le richieste ancora attive e di selezionare una richiesta per assegnarsela. Se dopo 2 minuti dalla richiesta nessun tassista ha accettato, la richiesta viene marcata come scaduta.

Al termine della corsa il tassista indica l'importo mostrato dal tassametro. Il sistema calcolerà automaticamente la durata complessiva della corsa.

I gestori del servizio ricevono una commissione pari al 3% dell'importo di tutte le corse pagate ai tassisti. Per questo motivo, possono generare un report che riporti, in forma aggregata per ciascun

tassista: quante corse sono state accettate, l'importo complessivo del guadagno del tassista, l'importo complessivo della commissione associato al tassista. Per le corse per cui non è ancora stata riscossa la commissione, il sistema calcola la commissione del 3%. I gestori possono indicare che l'importo è stato riscosso dalla carta di credito del tassista.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Tassista	Conducente registrato, mette a disposizione il veicolo	Autista	Veicolo, Corsa
Veicolo	Mezzo registrato, ha targa e capacità		Tassista
Cliente	Persona registrata che richiede corse	Utente	Richiesta, Corsa
Richiesta	Trasporto in attesa di accettazione	Richiesta di corsa	Cliente, Corsa
Corsa	Trasporto effettuato post-accettazione, importo e durata		Richiesta, Tassista, Commissione
Gestore	Operatore: genera report, registra riscossioni		Commissione
Commissione	Commissione pari al 3% dell'importo della corsa pagata al tassista; la riscossione è rappresentata dal flag riscossa della corsa.	Compenso, percentuale 3%	Corsa, Tassista, Gestore
Report	Prospetto generato dal sistema per ciascun tassista, contenente numero di corse accettate, guadagno totale e commissione totale		Tassista, Corsa, Commissione

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Tassista
- Ciascun tassista può liberamente iscriversi al servizio, mettendo a propria disposizione il proprio veicolo. - All'atto dell'iscrizione al servizio, il tassista deve registrare il proprio nome e cognome, il numero di patente, la targa del proprio veicolo ed il numero di carta di credito con cui pagare il costo del servizio. Deve inoltre comunicare la capacità del veicolo. - Un tassista ha la possibilità di visualizzare tutte le richieste ancora attive e di selezionare una richiesta per assegnarsela. - Al termine della corsa il tassista indica l'importo mostrato dal tassametro.

Frase relative a Cliente

- I clienti che utilizzano il sistema, all'atto della registrazione, devono fornire il proprio nome e cognome, un numero di telefono ed un numero di carta di credito per poter pagare le corse.
- Quando hanno bisogno di un taxi, i clienti indicano l'indirizzo di partenza e l'indirizzo di destinazione.

Frase relative a Veicolo

- Ciascun tassista può liberamente iscriversi al servizio, mettendo a propria disposizione il proprio veicolo.
- All'atto dell'iscrizione al servizio, il tassista deve registrare il proprio nome e cognome, il numero di patente, la targa del proprio veicolo ed il numero di carta di credito con cui pagare il costo del servizio. Deve inoltre comunicare la capacità del veicolo.

Frase relative a Richiesta

Un tassista ha la possibilità di visualizzare tutte le richieste ancora attive e di selezionare una richiesta per assegnarsela. Se dopo 2 minuti dalla richiesta nessun tassista ha accettato, la richiesta viene marcata come scaduta.

Frase relative a Corsa

- Al termine della corsa il tassista indica l'importo mostrato dal tassametro. Il sistema calcolerà automaticamente la durata complessiva della corsa.
- I gestori del servizio ricevono una commissione pari al 3% dell'importo di tutte le corse pagate ai tassisti.
- Per le corse per cui non è ancora stata riscossa la commissione, il sistema calcola la commissione del 3%.

Frase relative a Commissione

- I gestori del servizio ricevono una commissione pari al 3% dell'importo di tutte le corse pagate ai tassisti.
- Per le corse per cui non è ancora stata riscossa la commissione, il sistema calcola la commissione del 3%.

Frase relative a Gestore

- I gestori del servizio ricevono una commissione pari al 3% dell'importo di tutte le corse pagate ai tassisti. Per questo motivo, possono generare un report che riporti, in forma aggregata per ciascun tassista: quante corse sono state accettate, l'importo complessivo del guadagno del tassista, l'importo complessivo della commissione associato al tassista.
- I gestori possono indicare che l'importo è stato riscosso dalla carta di credito del tassista.

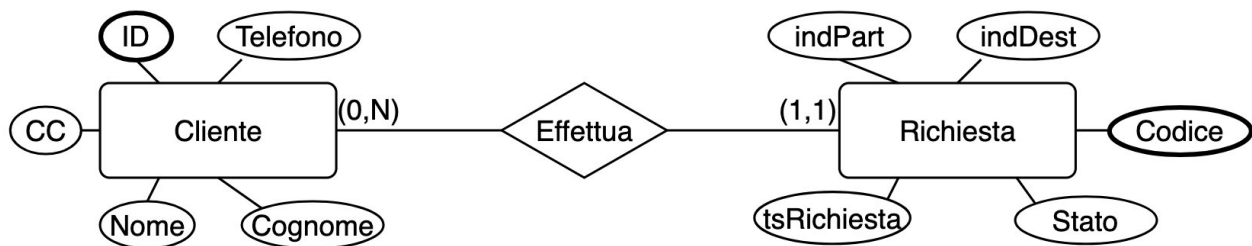
Frasi relative a Report

Per questo motivo, possono (i gestori) generare un report che riporti, in forma aggregata per ciascun tassista: quante corse sono state accettate, l'importo complessivo del guadagno del tassista, l'importo complessivo della commissione associato al tassista.

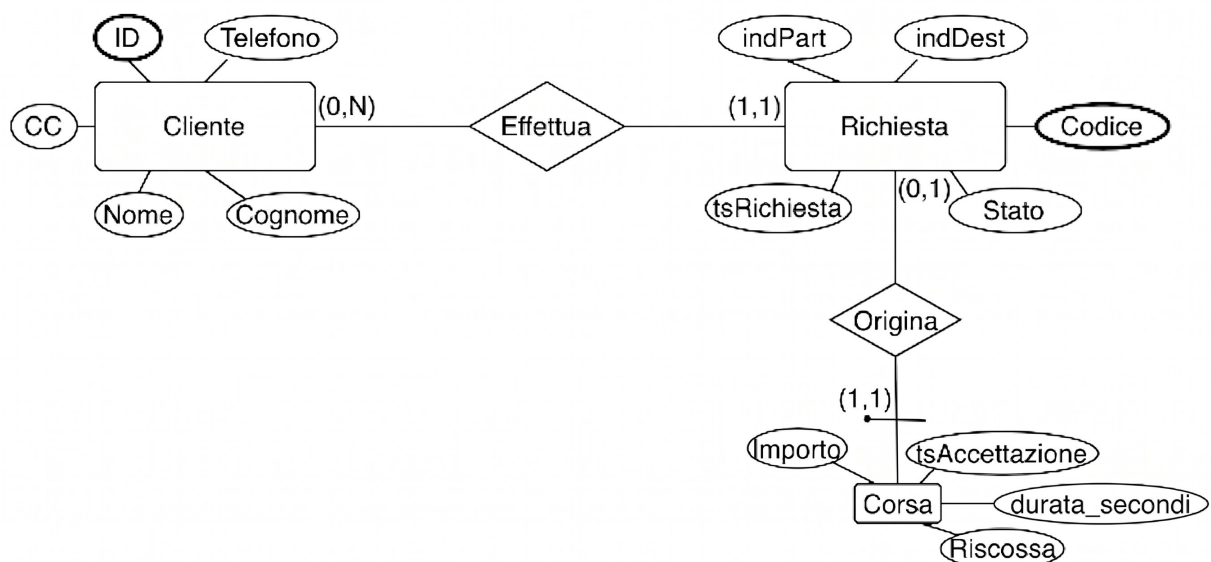
3. Progettazione concettuale

Costruzione dello schema E-R

Per la costruzione del modello ER si è adoperato un approccio a macchia d'olio:

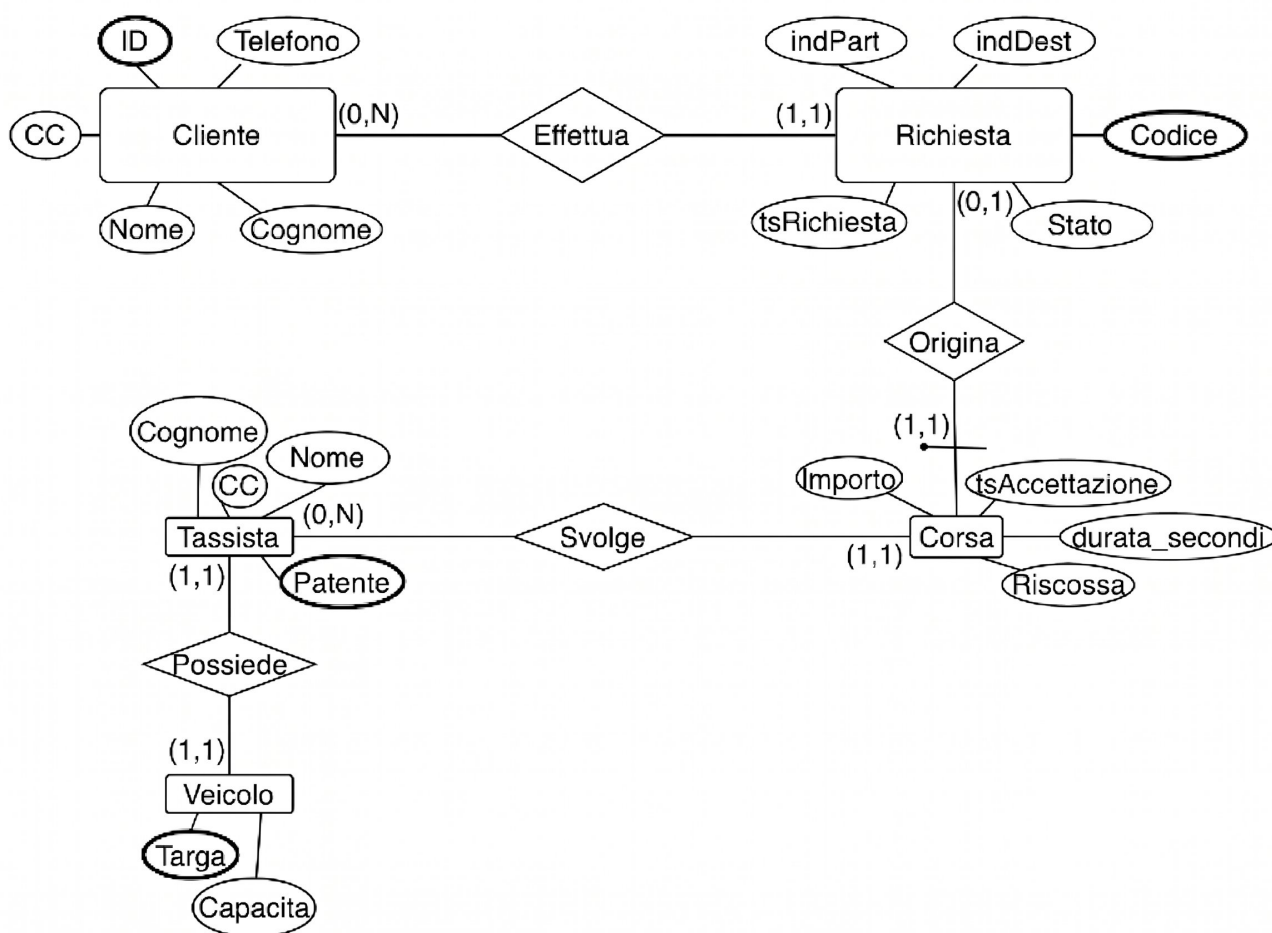


Si parte identificando la relazione più centrale del dominio: il cliente effettua una richiesta. Si modellano le entità **Cliente** e **Richiesta** collegate dalla relazione **Effettua**. Il **Cliente** è identificato da un **ID** surrogato, necessario perché nome e cognome non garantiscono univocità, e possiede gli attributi anagrafici nome, cognome, telefono e CC. La **Richiesta** è identificata da un codice e possiede gli indirizzi di partenza e destinazione (**indPart**, **indDest**), il timestamp di creazione (**tsRichiesta**, necessario per la regola dei 2 minuti) e lo stato (attiva/scaduta). La cardinalità **(0,N)** lato **Cliente** indica che un cliente registrato può effettuare zero o più richieste nel tempo, mentre **(1,1)** lato **Richiesta** indica che ogni richiesta proviene da esattamente un cliente.



Si espande lo schema aggiungendo l'entità **Corsa**, collegata a **Richiesta** tramite la relazione **Origina** con cardinalità **(0,1)–(1,1)**: una richiesta può non generare mai una corsa (se scade) oppure generarne

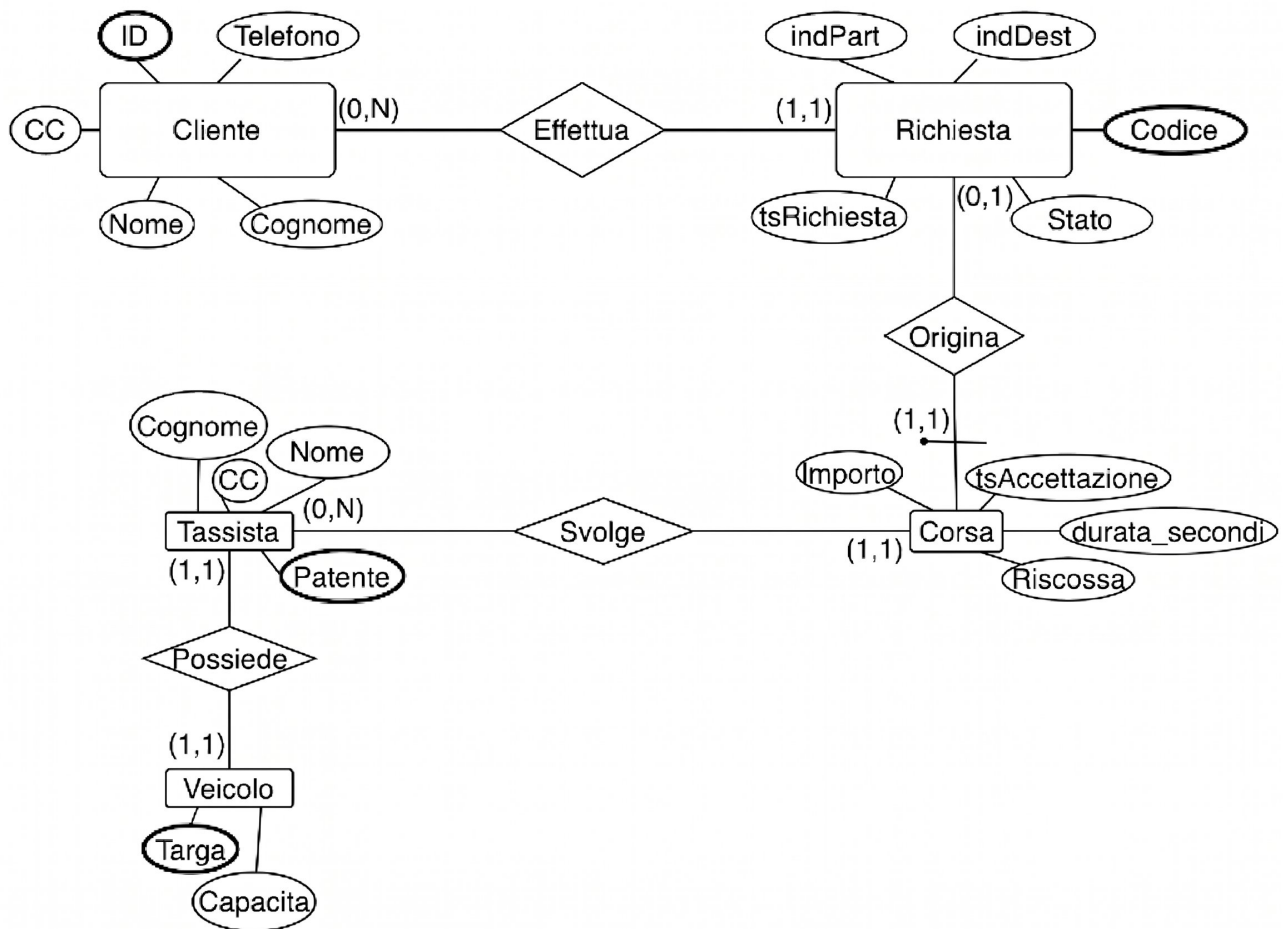
al massimo una, mentre ogni corsa nasce necessariamente da una richiesta. Corsa è modellata come entità debole poiché non possiede un identificatore proprio si identifica tramite il codice della richiesta da cui origina, dato il rapporto 1:1. I suoi attributi sono: tsAccettazione, importo, durata_secondi e riscossa. L'attributo tsAccettazione rappresenta il momento in cui il tassista prende in carico la corsa. L'attributo importo rappresenta il valore del tassametro registrato dal tassista al termine della corsa. L'attributo durata_secondi rappresenta la durata complessiva della corsa, calcolata automaticamente dal sistema al momento della registrazione della fine corsa. L'attributo riscossa indica se la commissione del 3% è stata riscossa dal gestore.



Si completa lo schema introducendo le entità Tassista e Veicolo. Il Tassista è identificato dalla patente e possiede nome, cognome e CC. La relazione Svolge collega Corsa a Tassista con cardinalità (1,1)–(0,N): ogni corsa è svolta da esattamente un tassista, mentre un tassista può aver svolto zero o più corse nel tempo. Il Veicolo è identificato dalla targa e possiede l'attributo capacità. La relazione Possiede collega Tassista a Veicolo con cardinalità (1,1)–(1,1), riflettendo l'assunzione derivata dall'uso del singolare nella specifica: ogni tassista registra un solo veicolo e ogni veicolo appartiene a un solo tassista.

Si è valutata la possibilità di introdurre una generalizzazione tra Cliente e Tassista, poiché entrambe le entità rappresentano soggetti registrati e condividono alcuni attributi anagrafici, come nome e cognome. Tuttavia si è scelto di mantenerle distinte, perché nella specifica esse compaiono come ruoli autonomi, partecipano ad associazioni differenti e possiedono attributi con significato operativo diverso. In particolare, la carta di credito del cliente è usata per il pagamento delle corse, mentre quella del tassista è usata per la riscossione della commissione. Inoltre una generalizzazione avrebbe richiesto l'introduzione di un identificatore comune del supertipo, rendendo meno immediato il ruolo della patente come identificatore del Tassista, che sarebbe diventata un ulteriore vincolo di unicità anziché il principale identificatore dell'entità. Sarebbe stato inoltre necessario stabilire se la specializzazione fosse esclusiva o sovrapposta, assumendo quindi se uno stesso soggetto possa essere contemporaneamente cliente e tassista, aspetto non specificato dal testo. Per questi motivi, la limitata riduzione di ridondanza non giustifica l'aumento di complessità concettuale.

Integrazione finale



L'attributo "CC" compare sia in Cliente sia in Tassista. In entrambi i casi rappresenta il numero di carta di credito, ma con ruoli diversi: per il cliente serve a pagare le corse, per il tassista serve al gestore per riscuotere la commissione. Non si tratta di un conflitto da risolvere: il nome è lo stesso perché il significato di base è lo stesso (coordinata bancaria), e le due entità sono distinte.

Regole aziendali

- Se entro 2 minuti dalla creazione di una richiesta (tsRichiesta) nessun tassista l'ha accettata, lo stato della richiesta viene automaticamente impostato a "scaduta".
- L'attributo stato dell'entità Richiesta può assumere esclusivamente i valori "attiva" o "scaduta".
- La commissione associata a ciascuna corsa pagata è sempre pari al 3% dell'importo della corsa.

- La durata complessiva della corsa viene calcolata automaticamente dal sistema al momento della registrazione della fine corsa. Il tassista inserisce soltanto l'importo mostrato dal tassametro.
- Un tassista può accettare esclusivamente richieste con stato attiva, non accettate da altri tassisti.
- Ogni tassista registra un solo veicolo.
- L'importo della corsa completata deve essere maggiore di zero.
- I gestori possono generare un report che riporti, per ciascun tassista: il numero di corse accettate, il guadagno complessivo e la commissione totale.
- Una corsa si considera conclusa quando importo e durata_secondi sono valorizzati.
- Una richiesta è visualizzabile nella lista delle richieste attive soltanto se ha stato attiva, non ha ancora originato alcuna corsa e appartiene alla finestra temporale dei due minuti dalla creazione.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Cliente	Persona registrata al sistema che effettua richieste di trasporto	nome, cognome, telefono, CC	ID
Richiesta	Richiesta di trasporto effettuata dal cliente, con indirizzo di partenza e destinazione; può essere attiva o scaduta dopo 2 minuti	indPart, indDest, tsRichiesta, stato	codice
Corsa	Trasporto effettivamente svolto a seguito dell'accettazione di una richiesta da parte di un tassista. Entità debole identificata tramite la richiesta da cui origina	tsAccettazione, importo, durata_secondi, riscossa	codice (ereditato da Richiesta)
Tassista	Conducente iscritto al servizio che mette a disposizione il proprio veicolo e svolge corse	nome, cognome, CC	patente
Veicolo	Mezzo di trasporto registrato dal tassista, caratterizzato da una capacità di posti	capacità	targa

4. Progettazione logica

Volume dei dati

Per stimare i volumi sono state fatte delle assunzioni ragionevoli su una società di taxi di medie dimensioni:

Volume dei dati: assunzioni

- Il sistema è operativo su una società di trasporto taxi di medie dimensioni
- Numero di tassisti attivi: 200, con un turnover del 20% all'anno (40 nuovi/anno)
- Ogni tassista svolge in media 15 corse al giorno su 250 giorni lavorativi all'anno
- Il 20% delle richieste scade senza essere accettata da alcun tassista
- Ogni cliente registrato effettua in media 10 richieste all'anno
- I dati vengono mantenuti nel sistema per un periodo di 5 anni

Calcoli:

Corse (orizzonte 5 anni):

$$\text{corse_totali} = 200 \text{ tassisti} \times 15 \text{ corse/giorno} \times 250 \text{ giorni/anno} \times 5 \text{ anni} = 3.750.000$$

Richieste (orizzonte 5 anni):

$$\text{richieste_totali} = \text{corse_totali} / 0.80 = 3.750.000 / 0.80 = 4.700.000 \text{ (arrotondato)}$$

di cui 3.750.000 accettate e ~940.000 scadute

Clienti:

$$\text{richieste_per_cliente_in_5_anni} = 10 \text{ richieste/anno} \times 5 \text{ anni} = 50$$

$$\text{clienti_totali} = 4.700.000 / 50 = 94.000$$

Tassisti:

Con turnover del 20%/anno su 5 anni: $200 + 40 \times 5 = 400$ (stimando che anche i tassisti cessati restano nel DB per storico)

Veicoli:

Un veicolo per tassista (dalla cardinalità (1,1)–(1,1) di Possiede): 400

Concetto nello schema	Tipo ¹	Volume atteso
Cliente	E	94.000
Richiesta	E	4.700.000
Corsa	E	3.750.000
Tassista	E	400
Veicolo	E	400

¹ Indicare con E le entità, con R le relazioni

Effettua	R	4.700.000
Origina	R	3.750.000
Svolge	R	3.750.000
Possiede	R	400

Tavola delle operazioni

Assunzioni operative:

- Ogni tassista consulta la lista richieste attive circa 20 volte al giorno lavorativo
- I gestori generano report con cadenza mensile
- Sia i tassisti che i clienti che i gestori si autenticano al sistema tramite credenziali. Username e password vengono quindi gestiti come dati tecnici di autenticazione, necessari per identificare correttamente chi esegue ciascuna operazione e per applicare i privilegi previsti dai ruoli applicativi.
- Ogni tassista effettua in media un login al giorno lavorativo (inizio turno)
- Ogni cliente effettua un login prima di creare una richiesta
- Turnover tassisti: 40 nuove registrazioni all'anno (derivato dalle assunzioni sul volume)

Calcoli delle frequenze:

- Registra cliente: 94.000 clienti in 5 anni / $(5 \times 365 \text{ giorni}) \approx 50/\text{giorno}$
- Crea richiesta: 3.750 richieste/giorno (dal volume)
- Registra tassista: 40/anno
- Lista richieste attive: 200 tassisti $\times$ 20 consultazioni = 4.000/giorno
- Accetta richiesta: 3.000 corse/giorno (coincide con le corse create)
- Registra fine corsa: 3.000/giorno (una per ogni corsa accettata)
- Genera report: stima 10/mese
- Segna riscossione: una per corsa, seguendo l'interpretazione letterale $\rightarrow$ 3.000/giorno
- Login: 200 tassisti + 3.750 clienti (uno per richiesta) + ~ 10 gestori $\approx$ 4.000/giorno

Cod.	Descrizione	Frequenza attesa
CL1	Registra cliente	50/gg
CL2	Crea richiesta	3.750/gg
TA1	Registra tassista	40/anno
TA2	Lista richieste attive	4.000/gg
TA3	Accetta richiesta	3.000/gg
TA4	Registra fine corsa	3.000/gg
GE1	Genera report	10/mese
GE2	Segna riscossione	3.000/gg
L1	Login	4.000/gg

Costo delle operazioni

CL1 — Registra cliente (50/giorno)

Entità coinvolte: Utenti, Cliente

Operazioni:

- Scrittura nuovo Utente: 1S
- Scrittura nuovo Cliente: 1S

$$\text{Costo unitario} = 0 + 2 \times 2 = 4$$

$$\text{Costo giornaliero} = 4 \times 50 = 200 \text{ accessi/giorno}$$

CL2 — Crea richiesta (3.750/giorno)

Entità coinvolte: Cliente, Richiesta, Effettua

Operazioni:

- Lettura Cliente: 1L
- Scrittura nuova Richiesta: 1S
- Scrittura relazione Effettua: 1S

$$\text{Costo unitario} = 1 + 2 \times 2 = 5$$

$$\text{Costo giornaliero} = 5 \times 3.750 = 18.750 \text{ accessi/giorno}$$

TA1 — Registra tassista (40/anno) Entità coinvolte: Utenti, Veicolo, Tassista, Possiede

Operazioni:

- Scrittura nuovo Tassista: 1S
- Scrittura nuovo Veicolo: 1S
- Scrittura nuovo Utente: 1S
- Scrittura relazione Possiede: 1S

$$\text{Costo unitario} = 0 + 2 \times 4 = 8$$

$$\text{Costo annuale} = 8 \times 40 = 320 \text{ accessi/anno} \approx 1 \text{ accesso/giorno}$$

TA2 — Lista richieste attive (4.000/giorno)

Entità coinvolte: Richiesta, Corsa

Operazioni:

- Lettura Richiesta: 1L
- Lettura Corsa: 1L

$$\text{Costo unitario} = 2 + 0 = 2$$

$$\text{Costo giornaliero} = 2 \times 4.000 = 8.000 \text{ accessi/giorno}$$

TA3 — Accetta richiesta (3.000/giorno)

Entità coinvolte: Richiesta, Corsa, Origina, Tassista, Svolge

- Operazioni:
- Lettura Richiesta per verificare stato e bloccare la richiesta: 1L
- Lettura Origina: 1L
- Scrittura nuova Corsa: 1S
- Scrittura relazione Origina: 1S
- Lettura Tassista: 1L
- Scrittura relazione Svolge: 1S

Costo unitario = $3 + 2 \times 3 = 9$

Costo giornaliero = $9 \times 3.000 = 27.000$ accessi/giorno

TA4 — Registra fine corsa (3.000/giorno)

Entità coinvolte: Corsa

Operazioni:

- Lettura Corsa: 1L
- Scrittura Corsa: 1S

Costo unitario = $1 + 2 \times 1 = 3$

Costo giornaliero = $3 \times 3.000 = 9.000$ accessi/giorno

GE1 — Genera report per tassista (10/mese)

Entità coinvolte: Tassista, Corsa, Svolge

Operazioni:

- Lettura Tassista: 1L
- Lettura Corsa: 1L
- Lettura Svolge: 1L

Costo unitario = $3 + 0 = 3$

Costo mensile = $3 \times 10 = 30$ accessi/mese ≈ 1 accesso/giorno

GE2 — Segna riscossione (3.000/giorno)

Entità coinvolte: Corsa

Operazioni:

- Lettura Corsa: 1L
- Scrittura Corsa: 1S

Costo unitario = $1 + 2 \times 1 = 3$

Costo giornaliero = $3 \times 3.000 = 9.000$ accessi/giorno

L1 — Login (4.000/giorno)

Entità coinvolte: Utenti

Operazioni:

- Lettura Utenti per verifica credenziali: 1L

Costo unitario = $1 + 0 = 1$

Costo giornaliero = $1 \times 4.000 = 4.000$ accessi/giorno

Nota: la tabella Utenti non fa parte dello schema E-R concettuale, ma è un'entità tecnica di supporto all'autenticazione. Verrà introdotta nella ristrutturazione del modello ER.

Ristrutturazione dello schema E-R

Dall'analisi dei costi delle operazioni emerge che le ottimizzazioni più convenienti non consistono nell'accorpamento delle principali entità del dominio, bensì nella traduzione delle relazioni con cardinalità massima 1. In particolare, Effettua e Svolge, rispettivamente con cardinalità $(0,N)-(1,1)$, vengono tradotte come chiavi esterne rispettivamente in Richiesta (verso Cliente) e in Corsa (verso Tassista), seguendo la regola standard di traduzione delle relazioni asimmetriche. La relazione Origina, con cardinalità $(0,1)-(1,1)$, viene trattata in modo peculiare: essendo Corsa un'entità debole identificata tramite Richiesta, la chiave primaria di Corsa coincide con la chiave di Richiesta, che funge contemporaneamente da FK verso di essa. Questo risultato è equivalente a quello che si otterrebbe traducendo Origina come FK, con l'ulteriore vantaggio di garantire strutturalmente il vincolo 1:1. Tali scelte riducono il costo delle operazioni più frequenti: per CL2 – Crea richiesta il costo unitario passa da 5 a 3 accessi (da 18.750 a 11.250 accessi/giorno), eliminando l'accesso separato alla struttura Effettua; analoghi risparmi si ottengono per TA3 – Accetta richiesta.

Al contrario, non risulta conveniente accorpare Richiesta e Corsa in un'unica entità, nonostante la relazione 1:1 tra richiesta accettata e corsa generata: circa il 20% delle richieste scade senza essere accettata e non genera una corsa, quindi l'accorpamento introdurrebbe numerosi campi NULL sui record corrispondenti e complicherebbe la distinzione logica tra richieste non accettate e corse effettivamente svolte. La separazione inoltre agevola l'operazione TA2 – Lista richieste attive, che interroga principalmente l'entità Richiesta e verifica l'assenza di una corsa associata, evitando al contempo l'introduzione di attributi nulli e mantenendo distinta la semantica tra richiesta e corsa effettivamente accettata.

La fusione tra Tassista e Veicolo, pur strutturalmente ammissibile data la cardinalità $(1,1)-(1,1)$ della relazione Possiede, non è giustificata dall'analisi dei costi. L'operazione TA1 – Registra tassista ha frequenza molto bassa (40/anno) e non esistono altre operazioni che richiedano accesso intensivo all'entità Veicolo, rendendo il guadagno prestazionale dell'accorpamento marginale rispetto alla perdita di chiarezza semantica del modello.

Per TA1 — Registra tassista, nella stima precedente alla ristrutturazione l'operazione coinvolge anche la relazione Possiede, oltre a Utenti, Tassista e Veicolo, per un costo unitario pari a 8. Dopo l'assorbimento di Possiede tramite la chiave esterna targa_veicolo in Tassista, l'operazione richiede soltanto l'inserimento in Utenti, Veicolo e Tassista, riducendo il costo unitario a 6. Tale ottimizzazione ha comunque impatto limitato, data la bassa frequenza dell'operazione.

In fase di progettazione fisica viene inoltre introdotta la tabella tecnica Utenti, non presente nello schema E-R concettuale. Clienti, tassisti e gestori accedono al sistema tramite credenziali personali. La tabella Utenti memorizza quindi username, password e ruolo applicativo. Per collegare le credenziali ai soggetti del dominio, le tabelle Cliente e Tassista contengono un riferimento univoco a Utenti(username).

Trasformazione di attributi e identificatori

L'unica omonimia rilevante riguarda l'attributo CC, presente sia in Cliente sia in Tassista. Tale coincidenza non genera un vero conflitto strutturale, perché nei due casi l'attributo appartiene a entità differenti e mantiene lo stesso significato generale di coordinata bancaria.

L'aspetto più importante riguarda invece gli identificatori esterni. L'entità Corsa è stata modellata come entità dipendente da Richiesta, poiché ogni corsa origina da una e una sola richiesta, mentre una richiesta può originare al più una corsa. Per facilitare la traduzione nello schema relazionale, l'identificatore esterno di Corsa viene trasformato nell'attributo CodiceRichiesta, ottenuto dalla chiave primaria di Richiesta. Tale attributo viene riportato nella relazione Corsa e ne costituisce contemporaneamente la chiave primaria e la chiave esterna verso Richiesta.

Anche le associazioni Effettua e Svolge, essendo associazioni binarie 1:N prive di attributi propri, vengono trasformate mediante assorbimento della chiave del lato uno nel lato molti. Di conseguenza: in Richiesta viene inserito l'attributo IDCliente, chiave esterna verso Cliente, in sostituzione della relazione Effettua; in Corsa viene inserito l'attributo PatenteTassista, chiave esterna verso Tassista, in sostituzione della relazione Svolge.

Per quanto riguarda Possiede, poiché collega Tassista e Veicolo con cardinalità 1:1 e partecipazione totale da entrambi i lati, non è necessario introdurre una relazione autonoma anche nello schema relazionale. Per semplificare la traduzione si assorbe l'associazione inserendo in Tassista l'attributo targa_veicolo, come chiave esterna verso Veicolo, imponendo su di esso un vincolo di unicità. In questo modo si mantiene separata l'entità Veicolo, ma si evita la creazione di una tabella dedicata alla sola associazione Possiede.

Traduzione di entità e associazioni

Cliente(id, username, nome, cognome, telefono, CC)

PK: id

FK: username → Utenti(username)

UQ: username

Richiesta(codice, indPart, indDest, tsRichiesta, stato, id_cliente)

PK: codice

FK: id_cliente → Cliente(id)

Corsa(codice_richiesta, tsAccettazione, importo, durata_secondi, riscossa, patente_tassista)

PK: codice_richiesta

FK: codice_richiesta → Richiesta(codice)

FK: patente_tassista → Tassista(patente)

Tassista(patente, username, nome, cognome, CC, targa_veicolo)

PK: patente

FK: username → Utenti(username)

FK: targa_veicolo → Veicolo(targa)

UQ: username

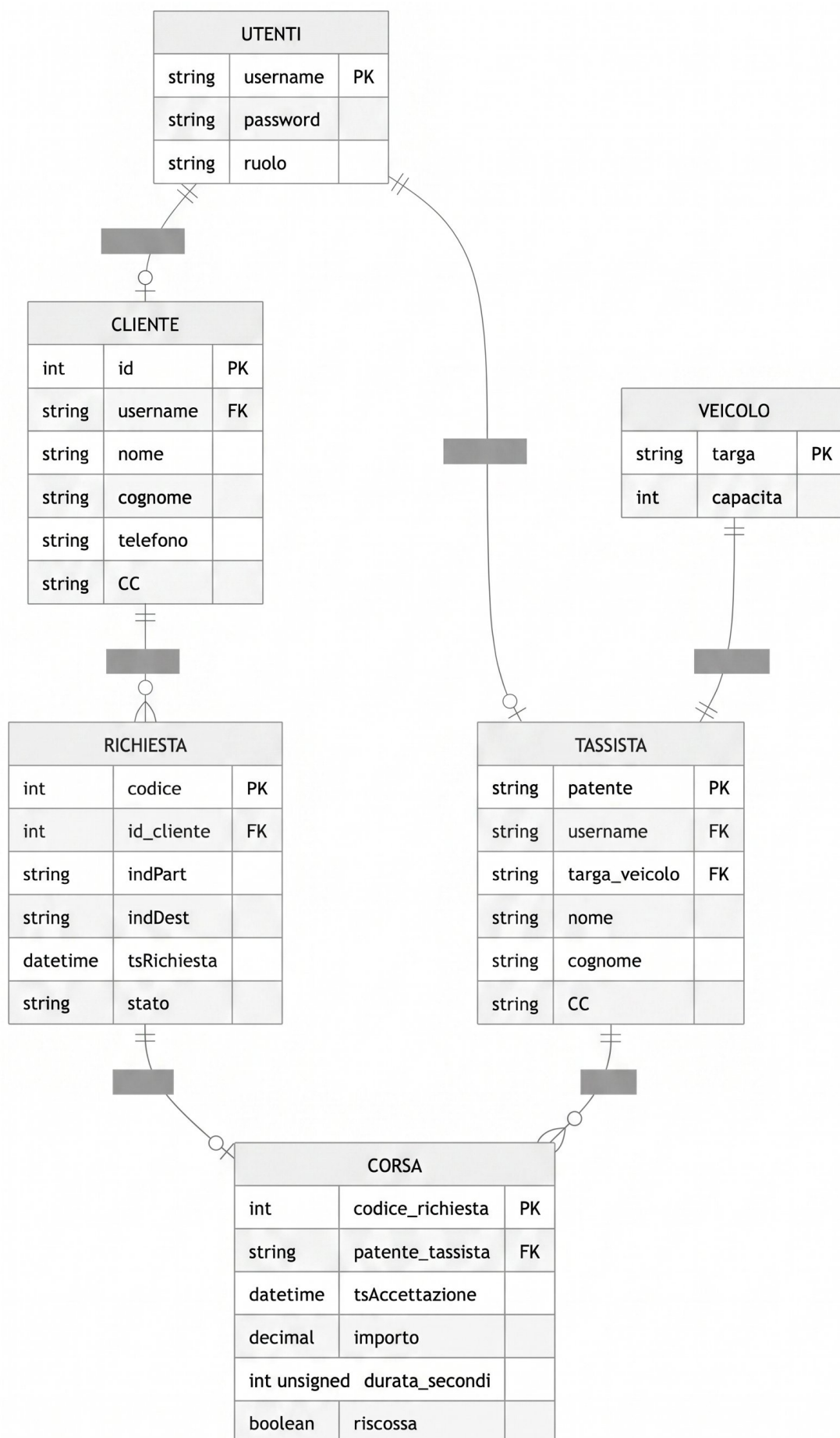
UQ: targa_veicolo

Veicolo(targa, capacità)

PK: targa

Utenti(username, password, ruolo)

PK: username



Normalizzazione del modello relazionale

Verifica la conformità alla BCNF per ciascuna tabella del modello:

Cliente(id, username, nome, cognome, telefono, CC):

Le dipendenze funzionali principali sono $id \rightarrow \text{username, nome, cognome, telefono, CC}$ e $\text{username} \rightarrow id, \text{nome, cognome, telefono, CC}$, poiché username è vincolato a essere unico. Entrambi i determinanti sono superchiavi, quindi la relazione è in BCNF.

Richiesta(codice, indPart, indDest, tsRichiesta, stato, id_cliente):

l'unica dipendenza funzionale non banale è $\text{codice} \rightarrow \text{indPart, indDest, tsRichiesta, stato, id_cliente}$.

Il determinante coincide con la chiave primaria. In BCNF.

Corsa(codice_richiesta, tsAccettazione, importo, durata_secondi, riscossa, patente_tassista):

l'unica dipendenza funzionale non banale è $\text{codice_richiesta} \rightarrow \text{tsAccettazione, importo, durata_secondi, riscossa, patente_tassista}$. Il determinante coincide con la chiave primaria. In BCNF.

Tassista(patente, username, nome, cognome, CC, targa_veicolo): Le dipendenze principali sono $\text{patente} \rightarrow \text{username, nome, cognome, CC, targa_veicolo}$, $\text{username} \rightarrow \text{patente, nome, cognome, CC, targa_veicolo}$ e $\text{targa_veicolo} \rightarrow \text{patente, username, nome, cognome, CC}$. I determinanti sono superchiavi, poiché patente è chiave primaria e username e targa_veicolo sono vincolati a essere unici.

Veicolo(targa, capacità):

l'unica dipendenza funzionale non banale è $\text{targa} \rightarrow \text{capacità}$. Il determinante coincide con la chiave primaria. In BCNF.

Utenti(username, password, ruolo):

l'unica dipendenza funzionale non banale è $\text{username} \rightarrow \text{password, ruolo}$. Il determinante coincide con la chiave primaria. In BCNF.

Tutte le relazioni del modello sono già in BCNF.

5. Progettazione fisica

Utenti e privilegi

Utente	Operazioni consentite
cliente	CL2 (Crea richiesta)
tassista	TA2 (Lista richieste attive), TA3 (Accetta richiesta), TA4 (Registra fine corsa)
gestore	GE1 (Genera report per tassista), GE2 (Segna riscossione)
login	L1 (Login)
registrazione	CL1 — Registra cliente, TA1 — Registra tassista

Le operazioni CL1 — Registra cliente e TA1 — Registra tassista sono procedure di registrazione iniziale e possono essere invocate prima del login. In questa fase vengono acquisiti sia i dati richiesti dalla specifica sia le credenziali scelte dall'utente, che vengono memorizzate nella tabella tecnica Utenti.

Dopo la registrazione, il cliente o il tassista può autenticarsi tramite L1 — Login e accedere alle sole operazioni previste dal proprio ruolo. Per questo motivo, le operazioni di registrazione sono associate a un contesto tecnico separato, denominato registrazione, mentre le operazioni successive sono riservate ai ruoli autenticati cliente, tassista e gestore.

In questa progettazione non vengono concessi privilegi diretti di SELECT, INSERT, DELETE o UPDATE sulle tabelle del database. Gli utenti possono eseguire soltanto le stored procedure autorizzate, in accordo con il principio del minimo privilegio.

Strutture di memorizzazione

Tabella Cliente		
Colonna	Tipo di dato	Attributi ²
id	INT	PK, AI, NN
nome	VARCHAR(50)	NN
cognome	VARCHAR(50)	NN
username	VARCHAR(50)	NN, UQ, FOREIGN KEY
telefono	VARCHAR(20)	NN
CC	VARCHAR(19)	NN

Tabella Richiesta

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Colonna	Tipo di dato	Attributi ³
codice	INT	PK, AI, NN
id_cliente	INT	NN, FOREIGN KEY
indPart	VARCHAR(200)	NN
indDest	VARCHAR(200)	NN
tsRichiesta	DATETIME	NOT NULL
stato	ENUM('attiva', 'scaduta')	NN, DEFAULT 'attiva'

Tabella Corsa		
Colonna	Tipo di dato	Attributi ⁴
codice_richiesta	INT	PK, NN, FOREIGN KEY
patente_tassista	VARCHAR(15)	NN, FOREIGN KEY
durata_secondi	INT UNSIGNED	NULL
importo	DECIMAL(8,2)	NULL
tsAccettazione	DATETIME	NN
riscossa	BOOLEAN	NN, DEFAULT FALSE

Tabella Tassista		
Colonna	Tipo di dato	Attributi ⁵
patente	VARCHAR(15)	PK, NN
targa_veicolo	VARCHAR(10)	NN, UQ, FOREIGN KEY
username	VARCHAR(50)	NN, UQ, FOREIGN KEY
nome	VARCHAR(50)	NN
cognome	VARCHAR(50)	NN
CC	VARCHAR(19)	NN

Tabella Veicolo		
Colonna	Tipo di dato	Attributi ⁶
targa	VARCHAR(10)	PK, NN
capacità	TINYINT UNSIGNED	NN

³ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁴ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁵ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁶ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella Utenti		
Colonna	Tipo di dato	Attributi ⁷
username	VARCHAR(50)	PK, NN
password	CHAR(32)	NN
ruolo	ENUM('tassista', 'cliente', 'gestore')	NN

Indici

Tabella Richiesta	
Indice <u>idx_richiesta_stato_ts</u>	Tipo ⁸ :
stato, tsRichiesta	IDX

Si introduce un indice secondario composto sugli attributi stato e tsRichiesta della tabella Richiesta per ottimizzare l'operazione TA2 — Lista richieste attive. La vista seleziona infatti le richieste con stato attiva e temporalmente ancora valide rispetto alla regola dei 2 minuti. L'indice consente di restringere rapidamente la ricerca alle richieste candidate, evitando scansioni complete della tabella Richiesta.

Trigger

Non sono stati introdotti trigger applicativi.

In questa progettazione si è scelto di mantenere la logica applicativa all'interno delle stored procedure, evitando di distribuirla nei trigger.

Gli utenti dell'applicazione non hanno privilegi diretti di SELECT, INSERT, UPDATE o DELETE sulle tabelle, ma possono soltanto eseguire le procedure autorizzate. In questo modo ogni operazione di scrittura passa necessariamente attraverso le stored procedure, dove vengono effettuati i controlli necessari.

I vincoli strutturali, come chiavi primarie, chiavi esterne, vincoli di unicità e valori non nulli, sono gestiti direttamente dal DBMS. I vincoli semantici, come capacità positiva del veicolo, importo positivo e riscossione di sole corse completate, vengono controllati nelle procedure corrispondenti. La durata della corsa viene calcolata automaticamente dalla procedura TA4 utilizzando il timestamp

⁷ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁸ IDX = index, UQ = unique, FT = full text, PR = primary.

corrente del DBMS e il valore di tsAccettazione già registrato al momento dell'accettazione della richiesta.

Eventi

Per rispettare in modo puntuale la regola secondo cui una richiesta deve essere marcata come scaduta dopo 2 minuti se non accettata si introduce un evento effimero per ogni richiesta creata. Tale evento viene istanziato dalla procedura CL2 – Crea richiesta subito dopo l'inserimento della nuova richiesta, viene schedulato a 2 minuti dalla creazione e viene eliminato automaticamente dopo l'esecuzione. Al momento dello scatto, l'evento marca la richiesta come scaduta solo se essa è ancora attiva e non ha originato alcuna corsa. Il codice seguente rappresenta il template dell'evento generato dinamicamente dalla procedura CL2 – Crea richiesta; il valore <codice> viene sostituito con il codice della richiesta appena inserita. L'istruzione SET GLOBAL event_scheduler = ON viene eseguita una sola volta in fase di configurazione del DBMS. Gli eventi effimeri vengono invece creati dinamicamente dalla procedura CL2.

```
SET GLOBAL event_scheduler = ON;
CREATE EVENT ev_scadenza_richiesta_<codice>
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 2 MINUTE
ON COMPLETION NOT PRESERVE
DO
UPDATE Richiesta r
SET r.stato = 'scaduta'
WHERE r.codice = <codice>
AND r.stato = 'attiva'
AND NOT EXISTS (
    SELECT 1
    FROM Corsa c
    WHERE c.codice_richiesta = r.codice
);
```

Viste

1. Vista delle richieste attive, supporta l'operazione TA2 – Lista richieste attive.

```
CREATE OR REPLACE VIEW vw_richieste_attive AS
SELECT
```

```
r.codice,  
r.indPart,  
r.indDest,  
r.tsRichiesta  
FROM Richiesta r  
WHERE r.stato = 'attiva'  
AND r.tsRichiesta > CURRENT_TIMESTAMP - INTERVAL 2 MINUTE  
AND NOT EXISTS (  
    SELECT 1  
    FROM Corsa c  
    WHERE c.codice_richiesta = r.codice  
);
```

La vista supporta l'operazione TA2 — Lista richieste attive. Restituisce soltanto le richieste ancora visualizzabili dai tassisti, cioè quelle con stato attiva, create da meno di due minuti e non ancora associate ad alcuna corsa. Il filtro temporale rende la vista coerente con la regola dei due minuti e consente di sfruttare l'indice composto su stato e tsRichiesta.

2. Vista per il report aggregato per tassista, l'operazione GE1 – Genera report per tassista.

```
CREATE OR REPLACE VIEW vw_report_tassisti AS  
SELECT  
    t.patente,  
    t.nome,  
    t.cognome,  
    COUNT(c.codice_richiesta) AS numero_corse,  
    COALESCE(SUM(c.importo), 0) AS guadagno_totale,  
    COALESCE(SUM(c.importo * 0.03), 0) AS commissione_totale  
FROM Tassista t  
LEFT JOIN Corsa c  
    ON c.patente_tassista = t.patente  
GROUP BY  
    t.patente,  
    t.nome,  
    t.cognome;
```

La vista supporta l'operazione GE1 — Genera report per tassista. Aggrega le corse per ciascun tassista e calcola il numero di corse accettate, il guadagno complessivo e la commissione totale pari al 3%

degli importi registrati. L'uso di LEFT JOIN consente di includere nel report anche tassisti che non hanno ancora svolto corse.

Stored Procedures e transazioni

L1 - Login

DELIMITER \$\$

DROP PROCEDURE IF EXISTS sp_login \$\$

CREATE PROCEDURE sp_login(

IN p_username VARCHAR(50),

IN p_password VARCHAR(50),

OUT p_ruolo INT

)

SQL SECURITY DEFINER

BEGIN

DECLARE v_ruolo VARCHAR(20) DEFAULT NULL;

DECLARE CONTINUE HANDLER FOR NOT FOUND

SET v_ruolo = NULL;

SELECT ruolo

INTO v_ruolo

FROM Utenti

WHERE username = p_username

AND password = MD5(p_password);

IF v_ruolo IS NULL THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Username o password non validi';

END IF;

SET p_ruolo = CASE v_ruolo

WHEN 'cliente' THEN 1

```
    WHEN 'tassista' THEN 2
    WHEN 'gestore' THEN 3
END;
END $$
```

DELIMITER ;

La procedura verifica le credenziali applicative confrontando username e hash MD5 della password con la tabella Utenti. Se non trova alcun utente valido, genera un errore applicativo tramite SIGNAL. In caso di login corretto, restituisce al client un codice numerico che identifica il ruolo dell'utente. Non è necessaria una transazione perché la procedura esegue una sola lettura e non modifica dati.

CL1 - Registra cliente

DELIMITER \$\$

```
DROP PROCEDURE IF EXISTS sp_registra_cliente $$
```

```
CREATE PROCEDURE sp_registra_cliente(
```

```
    IN p_username VARCHAR(50),
```

```
    IN p_password VARCHAR(50),
```

```
    IN p_nome VARCHAR(50),
```

```
    IN p_cognome VARCHAR(50),
```

```
    IN p_telefono VARCHAR(20),
```

```
    IN p_CC VARCHAR(19),
```

```
    OUT p_id_cliente INT
```

```
)
```

```
SQL SECURITY DEFINER
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
INSERT INTO Utenti(username, password, ruolo)
VALUES (p_username, MD5(p_password), 'cliente');
```

```
INSERT INTO Cliente(username, nome, cognome, telefono, CC)
VALUES (p_username, p_nome, p_cognome, p_telefono, p_CC);
```

```
SET p_id_cliente = LAST_INSERT_ID();
```

```
COMMIT;
END $$
```

```
DELIMITER ;
```

La procedura registra un nuovo cliente inserendo una riga in Utenti e una riga in Cliente. L'operazione è transazionale perché i due inserimenti devono essere atomici: in caso di errore, l'handler esegue ROLLBACK ed evita la creazione di utenti privi della corrispondente riga cliente. La password viene memorizzata come hash MD5.

TA1 - Registra tassista

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS sp_registra_tassista $$
```

```
CREATE PROCEDURE sp_registra_tassista(
```

```
    IN p_username VARCHAR(50),
```

```
    IN p_password VARCHAR(50),
```

```
    IN p_patente VARCHAR(15),
```

```
    IN p_nome VARCHAR(50),
```

```
    IN p_cognome VARCHAR(50),
```

```
    IN p_CC VARCHAR(19),
```

```
    IN p_targa VARCHAR(10),
```

```
    IN p_capacita INT
```

```
)
```

```
SQL SECURITY DEFINER
```

```
BEGIN
```



```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
IF p_capacita IS NULL OR p_capacita <= 0 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'La capacità del veicolo deve essere maggiore di zero';
```

```
END IF;
```

```
INSERT INTO Utenti(username, password, ruolo)
```

```
VALUES (p_username, MD5(p_password), 'tassista');
```

```
INSERT INTO Veicolo(targa, capacita)
```

```
VALUES (p_targa, p_capacita);
```

```
INSERT INTO Tassista(patente, username, nome, cognome, CC, targa_veicolo)
```

```
VALUES (p_patente, p_username, p_nome, p_cognome, p_CC, p_targa);
```

```
COMMIT;
```

```
END $$
```

```
DELIMITER ;
```

La procedura registra un nuovo tassista creando l'utente applicativo, il veicolo e il tassista. La transazione è necessaria perché i tre inserimenti rappresentano un'unica operazione logica. Il controllo sulla capacità del veicolo è gestito nella procedura.

CL2 - Crea richiesta

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS sp_crea_richiesta $$
```

```
CREATE PROCEDURE sp_crea_richiesta(  
    IN p_username_sessione VARCHAR(50),  
    IN p_indPart VARCHAR(200),  
    IN p_indDest VARCHAR(200),  
    OUT p_codice_richiesta INT  
)  
SQL SECURITY DEFINER  
BEGIN  
    DECLARE v_id_cliente INT;  
    DECLARE v_ts_richiesta DATETIME;  
    DECLARE v_ts_scadenza DATETIME;  
    DECLARE v_event_name VARCHAR(64);  
    DECLARE v_not_found BOOLEAN DEFAULT FALSE;  
  
    BEGIN  
        DECLARE CONTINUE HANDLER FOR NOT FOUND  
            SET v_not_found = TRUE;  
  
        DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        BEGIN  
            ROLLBACK;  
            RESIGNAL;  
        END;  
  
        SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
        START TRANSACTION;  
  
        SET v_not_found = FALSE;  
  
        SELECT c.id  
        INTO v_id_cliente  
        FROM Cliente c  
        JOIN Utenti u  
            ON u.username = c.username
```

```
WHERE c.username = p_username_sessione
AND u.ruolo = 'cliente';

IF v_not_found THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cliente non valido';
END IF;

SET v_ts_richiesta = CURRENT_TIMESTAMP;
SET v_ts_scadenza = v_ts_richiesta + INTERVAL 2 MINUTE;

INSERT INTO Richiesta(indPart, indDest, tsRichiesta, stato, id_cliente)
VALUES (p_indPart, p_indDest, v_ts_richiesta, 'attiva', v_id_cliente);

SET p_codice_richiesta = LAST_INSERT_ID();

COMMIT;
END;

SET v_event_name = CONCAT('ev_scadenza_richiesta_', p_codice_richiesta);

SET @sql_evento = CONCAT(
    'CREATE EVENT ', v_event_name, ' ',
    'ON SCHEDULE AT "', DATE_FORMAT(v_ts_scadenza, '%Y-%m-%d %H:%i:%s'), '" ',
    'ON COMPLETION NOT PRESERVE ',
    'DO UPDATE Richiesta r ',
    'SET r.stato = "scaduta" ',
    'WHERE r.codice = ', p_codice_richiesta, ' ',
    'AND r.stato = "attiva" ',
    'AND NOT EXISTS (',
    'SELECT 1 FROM Corsa c ',
    'WHERE c.codice_richiesta = r.codice',
    ') '
);
```

```
PREPARE stmt_evento FROM @sql_evento;  
EXECUTE stmt_evento;  
DEALLOCATE PREPARE stmt_evento;  
END $$
```

DELIMITER ;

La procedura crea una nuova richiesta per il cliente autenticato. La prima parte è transazionale: verifica il cliente, inserisce la richiesta con stato attiva e restituisce il codice generato. Dopo il COMMIT viene creato dinamicamente un evento effimero che, dopo due minuti, marca la richiesta come scaduta se non è stata accettata. La creazione dell'evento è posta dopo il COMMIT perché CREATE EVENT è un'istruzione DDL e non una normale operazione DML rollbackabile.

TA2 - Lista richieste attive

```
DELIMITER $$  
DROP PROCEDURE IF EXISTS sp_lista_richieste_attive $$  
CREATE PROCEDURE sp_lista_richieste_attive()  
SQL SECURITY DEFINER  
BEGIN  
    SELECT  
        codice,  
        indPart,  
        indDest,  
        tsRichiesta  
    FROM vw_richieste_attive  
    ORDER BY tsRichiesta;  
END $$
```

DELIMITER ;

La procedura espone al tassista il contenuto della vista vw_richieste_attive, ordinando le richieste per timestamp di creazione. Non usa una transazione esplicita perché esegue una sola interrogazione in lettura. La logica di selezione delle richieste visualizzabili è concentrata nella vista.

TA3 - Accetta richiesta

DELIMITER \$\$

DROP PROCEDURE IF EXISTS sp_accetta_richiesta \$\$

CREATE PROCEDURE sp_accetta_richiesta(

IN p_username_sessione VARCHAR(50),

IN p_codice_richiesta INT

)

SQL SECURITY DEFINER

BEGIN

DECLARE v_patente VARCHAR(15);

DECLARE v_stato VARCHAR(20);

DECLARE v_not_found BOOLEAN DEFAULT FALSE;

DECLARE CONTINUE HANDLER FOR NOT FOUND

SET v_not_found = TRUE;

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

ROLLBACK;

RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT t.patente

INTO v_patente

FROM Tassista t

JOIN Utenti u

ON u.username = t.username

WHERE t.username = p_username_sessione

AND u.ruolo = 'tassista';

IF v_not_found THEN

```
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT = 'Tassista non valido';  
END IF;
```

```
SET v_not_found = FALSE;
```

```
SELECT r.stato  
INTO v_stato  
FROM Richiesta r  
WHERE r.codice = p_codice_richiesta  
FOR UPDATE;
```

```
IF v_not_found THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Richiesta non esistente';  
END IF;
```

```
IF v_stato = 'scaduta' THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'La richiesta è scaduta';  
END IF;
```

```
IF EXISTS (  
    SELECT 1  
    FROM Corsa c  
    WHERE c.codice_richiesta = p_codice_richiesta  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'La richiesta è già stata accettata';  
END IF;
```

```
INSERT INTO Corsa(codice_richiesta, patente_tassista, tsAccettazione, importo, durata_secondi,  
riscossa)  
VALUES (p_codice_richiesta, v_patente, CURRENT_TIMESTAMP, NULL, NULL, FALSE);
```

```
COMMIT;  
END $$
```

```
DELIMITER ;
```

La procedura permette a un tassista di accettare una richiesta. È transazionale perché l'accettazione è un'operazione concorrente: più tassisti potrebbero tentare di accettare la stessa richiesta. La SELECT sulla richiesta usa FOR UPDATE per bloccare la riga fino al COMMIT, evitando accettazioni simultanee. La procedura verifica inoltre che il tassista sia valido, che la richiesta non sia scaduta e che non sia già stata accettata.

TA4 - Registra fine corsa

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS sp_registra_fine_corsa $$
```

```
CREATE PROCEDURE sp_registra_fine_corsa(  
    IN p_username_sessione VARCHAR(50),  
    IN p_codice_richiesta INT,  
    IN p_importo DECIMAL(8,2)  
)  
SQL SECURITY DEFINER  
BEGIN
```

```
    DECLARE v_patente VARCHAR(15);
```

```
    DECLARE v_patente_corsa VARCHAR(15);
```

```
    DECLARE v_ts_accettazione DATETIME;
```

```
    DECLARE v_durata_secondi INT UNSIGNED;
```

```
    DECLARE v_not_found BOOLEAN DEFAULT FALSE;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND  
        SET v_not_found = TRUE;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
        ROLLBACK;
```

```
    ROLLBACK;
```

```
RESIGNAL;  
END;  
  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;  
  
IF p_importo IS NULL OR p_importo <= 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Importo della corsa non valido';  
END IF;  
  
SELECT t.patente  
INTO v_patente  
FROM Tassista t  
JOIN Utenti u  
    ON u.username = t.username  
WHERE t.username = p_username_sessione  
    AND u.ruolo = 'tassista';  
  
IF v_not_found THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Tassista non valido';  
END IF;  
  
SET v_not_found = FALSE;  
  
SELECT c.patente_tassista, c.tsAccettazione, c.durata_secondi  
INTO v_patente_corsa, v_ts_accettazione, v_durata_secondi  
FROM Corsa c  
WHERE c.codice_richiesta = p_codice_richiesta  
FOR UPDATE;  
  
IF v_not_found THEN  
    SIGNAL SQLSTATE '45000'
```



```
SET MESSAGE_TEXT = 'Corsa non esistente';  
END IF;
```

```
IF v_patente_corsa <> v_patente THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Corsa non associata al tassista autenticato';  
END IF;
```

```
IF v_durata_secondi IS NOT NULL THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Corsa già conclusa';  
END IF;
```

```
UPDATE Corsa  
SET importo = p_importo,  
    durata_secondi = TIMESTAMPDIFF(SECOND, v_ts_accettazione, CURRENT_TIMESTAMP)  
WHERE codice_richiesta = p_codice_richiesta;
```

```
COMMIT;  
END $$
```

```
DELIMITER ;
```

La procedura registra la fine di una corsa accettata. Verifica che il tassista autenticato sia quello associato alla corsa, che l'importo sia positivo e che la corsa non sia già conclusa. La riga della corsa viene letta con FOR UPDATE per impedire aggiornamenti concorrenti. Al termine, la procedura salva l'importo e calcola automaticamente durata_secondi come differenza tra il timestamp corrente del DBMS e tsAccettazione.

GE1 - Genera-report

```
DELIMITER $$  
DROP PROCEDURE IF EXISTS sp_genera_report_tassisti $$  
CREATE PROCEDURE sp_genera_report_tassisti()  
SQL SECURITY DEFINER  
BEGIN
```

```
SELECT
    patente,
    nome,
    cognome,
    numero_corse,
    guadagno_totale,
    commissione_totale
FROM vw_report_tassisti
ORDER BY cognome, nome, patente;
END $$
```

DELIMITER ;

La procedura restituisce il report aggregato dei tassisti leggendo dalla vista vw_report_tassisti. Non richiede una transazione esplicita perché effettua una sola lettura e non modifica dati. L'ordinamento per cognome, nome e patente rende il risultato stabile e leggibile.

GE2 - Segna riscossione

DELIMITER \$\$

```
DROP PROCEDURE IF EXISTS sp_segna_riscossione $$
CREATE PROCEDURE sp_segna_riscossione(
    IN p_codice_richiesta INT
)
SQL SECURITY DEFINER
BEGIN
    UPDATE Corsa
    SET riscossa = TRUE
    WHERE codice_richiesta = p_codice_richiesta
        AND importo IS NOT NULL
        AND durata_secondi IS NOT NULL
        AND riscossa = FALSE;

    IF ROW_COUNT() = 0 THEN
        SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Corsa non riscuotibile';  
  END IF;  
END $$
```

DELIMITER ;

La procedura marca come riscossa la commissione relativa a una corsa. L'operazione è implementata con un singolo UPDATE condizionato, quindi è atomica senza bisogno di una transazione esplicita. La riscossione è consentita solo per corse concluse, con importo e durata_secondi valorizzati, e non ancora rimosse. Se nessuna riga viene aggiornata, viene generato l'errore "Corsa non riscuotibile".